

TFM: SLM with Transformers and GRU

Guillem Gonzalo Silveira

April 2026

c. 2161 words

Abstract

This thesis explores the engineering lifecycle of developing a Small Language Model (SLM) from scratch, utilizing a hybrid architecture of Transformers and Gated Recurrent Units (GRU). In a landscape dominated by massive models, the primary barrier for custom development is the extreme economic cost. This research shifts the focus from model scale to the complete deployment pipeline. We document the implementation of CUDA-optimized code, orchestration on AWS EC2 environments, and data strategies for conversational agents. Furthermore, we integrate Retrieval-Augmented Generation (RAG) to provide the model with external, up-to-date information, significantly reducing hallucinations without the prohibitive cost of full-scale retraining. Through performance and cost-benefit analysis using spot instances, we evaluate the feasibility of local development versus cloud scaling. Finally, we demonstrate production readiness by deploying the system on Kubernetes, visualized and managed through ArgoCD.

Abstract

Este trabajo de fin de máster investiga el ciclo de vida de ingeniería en el desarrollo de un Small Language Model (SLM) desde cero, integrando arquitecturas Transformer y GRU. En un entorno donde el desarrollo de modelos de lenguaje está limitado por altos costes operativos, este proyecto se centra en el flujo completo de ingeniería más allá de la escala del modelo. La investigación abarca el desarrollo de código optimizado para CUDA, la gestión de infraestructura en AWS EC2 y la preparación de datos para agentes conversacionales. Se incluye además la implementación de tecnologías RAG (Retrieval-Augmented Generation), que permiten al modelo consultar fuentes externas de información para responder con precisión sin necesidad de reentrenamientos costosos. Mediante el uso de instancias spot y subconjuntos de datos, se analizan los costes y el rendimiento del sistema. Finalmente, se valida la puesta en producción mediante un despliegue en Kubernetes gestionado con ArgoCD.

Abstract

Aquest treball de fi de màster investiga el cicle de vida d'enginyeria en el desenvolupament d'un Small Language Model (SLM) des de zero, integrant arquitectures Transformer i GRU. En un entorn on el desenvolupament de models de llenguatge està limitat per alts costos operatius, aquest projecte se centra en el flux complet d'enginyeria més enllà de l'escala del model. La investigació inclou el desenvolupament de codi optimitzat per a CUDA, la gestió d'infraestructura a AWS EC2 i la preparació de dades per a agents conversacionals. S'hi inclou a més la implementació de tecnologies RAG (Retrieval-Augmented Generation), que permeten al model consultar fonts externes d'informació per respondre amb precisió sense la necessitat de reentrenaments costosos. Mitjançant l'ús d'instàncies spot i subconjunts de dades, s'analitzen els costos i el rendiment del sistema. Finalment, es valida la posada en producció mitjançant un desplegament a Kubernetes gestionat amb ArgoCD.

Contents

1	Introducción	6
1.1	Motivación	6
1.2	Objetivos	6
2	Estado del Arte	7
2.1	Evolución de los Modelos de Lenguaje: De LLM a SLM	7
2.2	Limitaciones de los Transformers Puros	7
2.3	Arquitecturas Híbridas y Recurrencia Moderna	7
2.4	Aumento por Recuperación (RAG)	7
3	Fundamentos Teóricos	9
3.1	Transformers y el Mecanismo de Atención	9
3.2	Gated Recurrent Units (GRU)	9
3.3	Retrieval-Augmented Generation (RAG)	9
4	Diseño e Implementación de la Arquitectura Híbrida	10
4.1	Estrategia de Curación y Mezcla de Datos (Data Mixing)	10
4.1.1	Selección de Datasets	10
4.1.2	Generación de Datos de Dominio y RAG	10
4.1.3	Distribución del Dataset Final	10
4.2	Arquitectura del Modelo	11
4.3	Optimización con CUDA	11
5	Infraestructura Cloud y Análisis de Costes	12
5.1	Ecosistema AWS	12
5.2	Estrategia de Spot Instances	12
5.3	Evaluación de Rendimiento vs Coste	12
6	MLOps: Despliegue y Puesta en Producción	13
6.1	Contenedorización con Docker	13
6.2	Automatización de la Integración Continua (CI)	13
6.3	Gestión de Binarios y Control de Versiones	14
6.4	Orquestación con Kubernetes	14
6.5	GitOps con ArgoCD	14
6.6	Integración del Sistema RAG	14

7	Conclusiones y Trabajo Futuro	15
7.1	Viabilidad de los SLM Híbridos	15
7.2	Lecciones Aprendidas	15
7.3	Trabajo Futuro	15

1 Introducción

En la actualidad, los modelos de lenguaje de gran escala (LLM) han revolucionado la inteligencia artificial. Sin embargo, su desarrollo y entrenamiento están limitados a unas pocas organizaciones debido a los prohibitivos costes económicos y de computación. Este proyecto nace de la necesidad de explorar alternativas viables: los Small Language Models (SLM).

1.1 Motivación

La democratización de la IA requiere que las organizaciones puedan desarrollar, desplegar y controlar sus propios modelos sin depender de APIs de terceros o inversiones multimillonarias.

1.2 Objetivos

El objetivo principal es diseñar e implementar una arquitectura híbrida entre Transformers y GRU, analizando no solo su rendimiento teórico, sino todo el pipeline de ingeniería: desde el kernel CUDA y la contenedorización con Docker hasta el despliegue en Kubernetes mediante GitOps.

2 Estado del Arte

Este capítulo analiza el panorama actual de los modelos de lenguaje, la evolución hacia modelos más eficientes y las investigaciones previas en arquitecturas híbridas.

2.1 Evolución de los Modelos de Lenguaje: De LLM a SLM

En los últimos años, el campo del Procesamiento del Lenguaje Natural (NLP) ha estado dominado por los *Large Language Models* (LLM), cuya arquitectura base sigue siendo el Transformer propuesto por Vaswani et al. [1]. Modelos como Llama 2 [2] han demostrado capacidades emergentes sorprendentes al escalar el número de parámetros a miles de millones.

Sin embargo, recientemente ha surgido una tendencia hacia los *Small Language Models* (SLM), impulsada por la necesidad de eficiencia y despliegue en entornos con recursos limitados. Investigaciones como "*Textbooks Are All You Need*" [3] demuestran que la calidad de los datos puede compensar el tamaño del modelo, permitiendo que arquitecturas de menos de 3 mil millones de parámetros (como Phi-3 o Mistral) compitan en tareas específicas con modelos mucho más grandes. En este contexto, la familia de modelos SmolLM [4] representa el estado actual de la técnica en SLMs ultra-eficientes, optimizando tanto el proceso de entrenamiento como la curación de datos para maximizar el rendimiento en dispositivos de consumo.

2.2 Limitaciones de los Transformers Puros

A pesar de su éxito, los Transformers convencionales presentan un cuello de botella fundamental: la complejidad cuadrática del mecanismo de auto-atención respecto a la longitud de la secuencia [1]. Esto implica un consumo de memoria de video (VRAM) y un tiempo de cómputo que crece de forma prohibitiva para contextos largos. En un entorno de producción, esto se traduce en altos costes operativos y latencias elevadas, lo que motiva la búsqueda de arquitecturas con escalado lineal.

2.3 Arquitecturas Híbridas y Recurrencia Moderna

Para mitigar las limitaciones del Transformer, han resurgido las arquitecturas basadas en recurrencia, pero con enfoques modernos. Las *Gated Recurrent Units* (GRU) [5] ofrecen un manejo eficiente de secuencias con un estado oculto de tamaño fijo, eliminando el coste cuadrático.

Investigaciones recientes como Mamba [6] han propuesto modelos de espacio de estados (*State Space Models*) que logran un rendimiento comparable a los Transformers pero con inferencia de tiempo lineal. La tendencia actual, y el núcleo de este TFM, es la hibridación: combinar capas de atención para capturar relaciones globales con capas recurrentes o lineales para mantener la eficiencia secuencial.

2.4 Aumento por Recuperación (RAG)

Finalmente, el estado del arte no solo se centra en la arquitectura del modelo, sino en su interacción con fuentes externas. El *Retrieval-Augmented Generation* (RAG) [7] se

ha consolidado como el estándar para reducir alucinaciones en modelos de lenguaje. Esta técnica permite que el modelo consulte bases de datos externas en tiempo real, proporcionando respuestas basadas en evidencia sin necesidad de reentrenamientos costosos, un componente vital para asistentes técnicos especializados.

3 Fundamentos Teóricos

En este capítulo se analizan las tecnologías base que componen la propuesta híbrida.

3.1 Transformers y el Mecanismo de Atención

El impacto del mecanismo de *self-attention* en el procesamiento del lenguaje y su capacidad para capturar relaciones globales.

3.2 Gated Recurrent Units (GRU)

La eficiencia de las redes recurrentes en el manejo de secuencias y cómo las puertas de actualización permiten mantener una memoria selectiva con menor coste computacional que las LSTM.

3.3 Retrieval-Augmented Generation (RAG)

El concepto de aumentar la capacidad de respuesta del modelo mediante la recuperación de contexto externo, mitigando alucinaciones y proporcionando datos actualizados.

4 Diseño e Implementación de la Arquitectura Híbrida

Este capítulo detalla la aportación técnica central de este TFM: la integración de Transformers y GRU en un único modelo funcional, así como la estrategia de datos que permite su entrenamiento eficiente.

4.1 Estrategia de Curación y Mezcla de Datos (Data Mixing)

La calidad del entrenamiento de un Small Language Model (SLM) depende críticamente de la diversidad y representatividad de los datos. Se ha diseñado una estrategia de *Data Mixing* que equilibra la fluidez conversacional, el razonamiento lógico y el conocimiento de dominio específico.

4.1.1 Selección de Datasets

Se han seleccionado cuatro pilares de datos abiertos para dotar al modelo de capacidades generales:

- **OpenAssistant (OASST1):** Fuente principal de naturalidad, proporcionando turnos de diálogo con matices humanos reales.
- **ShareGPT:** Utilizado para transferir el estilo resolutivo y la estructura de respuesta de modelos avanzados.
- **Alpaca:** Aporta una base sólida de *instruction-tuning* para tareas directas.
- **UltraChat:** Fundamental para garantizar la consistencia en diálogos largos, poniendo a prueba la memoria de la arquitectura híbrida.

4.1.2 Generación de Datos de Dominio y RAG

Dado que el objetivo final es actuar como experto sobre la documentación de Zurich, se ha implementado un pipeline que convierte manuales técnicos en pares de Pregunta-Contexto-Respuesta. Esto entrena al modelo específicamente para la metodología **RAG**, enseñándole a priorizar la información recuperada externamente.

4.1.3 Distribución del Dataset Final

Para optimizar el rendimiento bajo restricciones de recursos, se ha definido la siguiente distribución de pesos:

Categoría	Dataset	Peso	Propósito
Dominio Específico	Sintético (Zurich)	50%	Conocimiento del stack técnico
Fluidez Humana	OpenAssistant	20%	Tono conversacional
Razonamiento	Magicode / WizardLM	20%	Lógica y <i>debugging</i>
Infraestructura	The Stack (YAML)	10%	Sintaxis DevOps

Table 1: Estrategia de mezcla de datos para el entrenamiento del SLM.

4.2 Arquitectura del Modelo

Descripción de la sinergia entre las capas de atención y las capas recurrentes. Se detalla cómo la GRU refina la salida de los bloques de atención para mejorar la eficiencia secuencial y reducir la carga computacional en el manejo de estados de memoria.

4.3 Optimización con CUDA

Desarrollo de kernels personalizados para maximizar el uso de la GPU. Se aborda el resto de paralelizar la parte de atención mientras se mantiene la eficiencia en la naturaleza secuencial de la GRU. El diseño de estos kernels se ha optimizado para manejar las longitudes de secuencia y el *padding* definidos en nuestra estrategia de datos, minimizando la latencia en el paso de estados entre la atención global y la recurrencia local.

5 Infraestructura Cloud y Análisis de Costes

Uno de los pilares de este trabajo es la demostración de la viabilidad económica del proyecto.

5.1 Ecosistema AWS

Uso de instancias EC2 optimizadas para computación acelerada.

5.2 Estrategia de Spot Instances

Análisis del uso de instancias spot para reducir drásticamente los costes de entrenamiento y pruebas.

5.3 Evaluación de Rendimiento vs Coste

Presentación de gráficas y métricas que comparan el tiempo de ejecución con el gasto económico real, demostrando la eficiencia de la arquitectura híbrida en entornos de recursos limitados.

6 MLOps: Despliegue y Puesta en Producción

El ciclo de vida del modelo culmina con su despliegue en un entorno de producción real. Para garantizar la portabilidad y consistencia entre entornos, se ha adoptado una estrategia de contenedorización avanzada.

6.1 Contenedorización con Docker

La empaquetación del modelo y su entorno de ejecución se realiza mediante Docker, utilizando un enfoque de construcción multi-etapa (*multi-stage build*) para optimizar el tamaño de la imagen final y mejorar la seguridad.

Se utiliza `uv` como gestor de paquetes de Python, lo que permite una resolución de dependencias y una instalación extremadamente rápidas en comparación con `pip`. El proceso se divide en dos fases:

- **Etapas de Construcción (*builder*):** Basada en una imagen ligera de `uv`, se encarga de sincronizar las dependencias y compilar el *bytecode* de Python, generando un entorno virtual aislado.
- **Etapas de Ejecución (*runtime*):** Utiliza una imagen mínima de Python 3.13-slim donde solo se copia el entorno virtual pre-construido y el código fuente del modelo, minimizando la superficie de ataque y el peso de la imagen. La ejecución se realiza mediante el punto de entrada `tfm-slm` gestionado por `uv`.

6.2 Automatización de la Integración Continua (CI)

Para automatizar el ciclo de vida de la imagen de Docker, se ha implementado un flujo de trabajo mediante GitHub Actions. Este proceso de integración continua garantiza que cada cambio en el código fuente sea validado y empaquetado automáticamente.

El flujo de trabajo definido realiza las siguientes acciones:

- **Disparadores:** Se activa ante cualquier *push* o *pull request* hacia la rama principal (`main`).
- **Seguridad y Autenticación:** Utiliza el token intrínseco de GitHub (`GITHUB_TOKEN`) para autenticarse de forma segura en el registro de contenedores sin necesidad de gestionar credenciales externas.
- **Gestión de Metadatos:** Emplea la acción `docker/metadata-action` para etiquetar las imágenes de forma inteligente, utilizando el hash del commit (`SHA`) y etiquetas de rama, lo que facilita la trazabilidad en el despliegue.
- **Optimización de Build:** Utiliza un sistema de caché persistente en GitHub Actions para reducir los tiempos de construcción, reutilizando capas de la imagen que no hayan sufrido cambios.

Esta automatización permite que el registro de contenedores de GitHub (`ghcr.io`) actúe como la única fuente de verdad para las imágenes que posteriormente serán orquestadas.

6.3 Gestión de Binarios y Control de Versiones

Dada la naturaleza del proyecto, que incluye tanto código fuente como el documento de la tesis en formato PDF, se ha implementado una política de gestión de archivos binarios mediante el uso de `.gitattributes`.

Se ha configurado específicamente el archivo `report/main.pdf` como binario puro. Esta medida evita que el sistema de control de versiones intente realizar fusiones de contenido (*merges*) sobre el PDF, lo que generaría conflictos irresolubles. En su lugar, se fuerza una política de sobrescritura, garantizando que la versión final del reporte sea siempre consistente y evitando la corrupción de datos en el historial de Git.

6.4 Orquestación con Kubernetes

Configuración del clúster de K8s para servir el modelo híbrido de forma escalable.

6.5 GitOps con ArgoCD

Implementación del flujo de despliegue continuo, permitiendo que cualquier cambio en la configuración se refleje automáticamente en el clúster.

6.6 Integración del Sistema RAG

Despliegue de la infraestructura de base de datos vectorial y su conexión con el modelo para la inferencia aumentada.

7 Conclusiones y Trabajo Futuro

Este capítulo resume los hallazgos del proyecto y las posibles líneas de investigación.

7.1 Viabilidad de los SLM Híbridos

Conclusiones sobre si un modelo de estas características es suficiente para casos de uso empresariales específicos.

7.2 Lecciones Aprendidas

Reflexión sobre los desafíos de ingeniería encontrados durante el desarrollo CUDA y el despliegue MLOps.

7.3 Trabajo Futuro

Posibilidades de mejora: cuantización del modelo, soporte para arquitecturas de inferencia como vLLM u Ollama, y fine-tuning con datos de dominio específico.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in neural information processing systems*, pp. 5998–6008, 2017.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [3] S. Gunasekar, Y. Zhang, J. Aneja, C. C. T. Mendes, A. Del Giorno, S. Gopi, M. Javaheripi, P. Kauffmann, G. de Rosa, O. Saarikivi, *et al.*, “Textbooks are all you need,” *arXiv preprint arXiv:2306.11644*, 2023.
- [4] L. B. Allal, A. Lozhkov, E. Bakouch, L. von Werra, and T. Wolf, “Smollm - blazingly fast and remarkably powerful.” <https://huggingface.co/blog/smollm>, 2024. Accessed: 2026-05-03.
- [5] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.
- [6] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.